



Super Mario Suika: Clon Arcade en C#

Estudiante: Mario Deusto Ríos

Ciclo Formativo: G.S. Desarrollo de Aplicaciones
Multiplataforma (DAM)

Curso Académico: 2025/2026

Tutor/a: Virginia Checa Calpena

ÍNDICE

1. INTRODUCCIÓN Y OBJETIVOS.....	3
1.1. Contexto y Motivación.....	3
1.2. Objetivos Generales.....	3
1.3. Objetivos Específicos.....	3
2. ANTECEDENTES Y ESTADO DEL ARTE.....	4
3. ANÁLISIS.....	5
3.1. Descripción detallada de la solución y flujo de trabajo.....	5
3.2. Requisitos Funcionales.....	5
3.3. Requisitos No Funcionales.....	5
3.4. Modelo de Datos Preliminar.....	5
4. DISEÑO.....	6
4.1. Diseño de los Módulos Principales.....	6
4.2 Metodología de Desarrollo.....	6
4.3. Arquitectura del sistema.....	7
4.4. Diagramas de casos de uso.....	7
4.5. Diagramas de Clases.....	8
4.6. Diseño de la Interfaz.....	9
5. RESULTADOS E IMPLEMENTACIÓN.....	10
6. CONCLUSIONES.....	11
6.1. Conclusiones del trabajo realizado.....	11
6.2. Trabajos futuros.....	11
7. BIBLIOGRAFÍA / WEBGRAFÍA.....	12

1. INTRODUCCIÓN Y OBJETIVOS

La memoria técnica a continuación recoge toda la planificación, desarrollo y resultados del proyecto, documentando la creación de un videojuego 2D desde cero.

1.1. Contexto y Motivación

El desarrollo de videojuegos de físicas con mecánicas de fusión ha experimentado un crecimiento exponencial en el mercado casual. El proyecto resuelve el reto técnico de programar un motor de físicas personalizado y un sistema de persistencia de datos utilizando únicamente las herramientas gráficas estándar de Windows (Windows Forms), sin recurrir a motores gráficos externos prefabricados como Unity o Godot. La motivación principal es consolidar los conocimientos de Programación Orientada a Objetos (POO), manejo de eventos y gestión de archivos locales.

1.2. Objetivos Generales

El objetivo global del sistema es diseñar, desarrollar y desplegar un videojuego arcade funcional, interactivo y optimizado, que permita a los usuarios interactuar con un entorno de físicas en tiempo real, garantizando la persistencia de sus récords y ofreciendo una experiencia audiovisual completa.

1.3. Objetivos Específicos

- **Desarrollar un motor de físicas propio:** Implementar cálculos de gravedad, fricción y colisiones circulares basadas en el Teorema de Pitágoras.
- **Implementar persistencia de datos:** Diseñar un sistema de lectura y escritura en archivos de texto plano (.txt) para almacenar un "Top 5" de puntuaciones históricas.
- **Gestión audiovisual:** Integrar un sistema de música de fondo en bucle y efectos de sonido reactivos a los eventos del juego.

- **Diseño de interfaz:** Desarrollar un menú de inicio intuitivo y un bucle de juego (Game Loop) estable a 60 fotogramas por segundo.

2. ANTECEDENTES Y ESTADO DEL ARTE

En la actualidad, el subgénero de puzzles físicos ha sido popularizado por títulos como Suika Game. Estos sistemas se basan en la acumulación de objetos en un contenedor cerrado, donde el contacto entre dos entidades idénticas genera una nueva entidad de mayor volumen, requiriendo gestión espacial por parte del usuario. Técnicamente, las soluciones previas suelen depender de librerías de físicas como Box2D. Este proyecto prescinde de dichas librerías para demostrar la capacidad de calcular colisiones e impulsos elásticos mediante matemáticas fundamentales aplicadas en C#.

3. ANÁLISIS

3.1. Descripción detallada de la solución y flujo de trabajo

El sistema presenta una pantalla de bienvenida donde el usuario puede consultar el "Top 5 Récor ds" históricos. Al iniciar la partida, el usuario controla un lanzador horizontal en la parte superior de la pantalla. El flujo de trabajo consiste en hacer clic para dejar caer "setas" de diferentes niveles (tamaños). El sistema procesa internamente las colisiones: si dos setas de igual nivel se tocan, se fusionan, suman puntos y generan un sonido, si una seta supera la línea límite superior, la partida finaliza, evaluando si la puntuación obtenida merece ser guardada en el archivo local de récor ds.

3.2. Requisitos Funcionales

- **Control de Lanzamiento:** El sistema debe permitir al usuario mover el proyectil horizontalmente con el ratón y soltarlo con un clic.
- **Mecánica de Fusión:** El sistema debe detectar colisiones y fusionar entidades del mismo nivel, incrementando la puntuación.
- **Gestión de Récor ds:** El sistema debe leer un archivo local al arrancar y sobrescribirlo al finalizar una partida si se alcanza una nueva puntuación máxima.
- **Herramientas de Testeo:** El sistema debe incluir atajos de teclado (Ctrl + G) para forzar el estado de "Game Over".

3.3. Requisitos No Funcionales

- **Rendimiento:** El bucle principal debe ejecutarse con un intervalo de 16 milisegundos para garantizar una tasa de refresco visual de 60 FPS.
- **Estabilidad:** El sistema debe evitar bloqueos al gestionar múltiples colisiones simultáneas mediante el uso de listas de creación y destrucción controladas.

3.4. Modelo de Datos Preliminar

- **Config:** Entidad estática que centraliza constantes físicas (gravedad, fricción), tamaños de pantalla y la lista temporal de récords.
- **GameData:** Entidad que almacena el estado actual de la partida (puntuación, setas activas, estado de finalización).
- **Seta:** Entidad que representa cada objeto físico (coordenadas X/Y, nivel, radio, velocidad vectorial).

4. DISEÑO

4.1. Diseño de los Módulos Principales

- **Módulo de Presentación (Partida.cs, Bienvenida.cs):** Gestiona la captura de eventos del teclado/ratón y el renderizado gráfico mediante el objeto Graphics y herramientas como Pen.
- **Módulo de Lógica y Físicas (GameService.cs):** Evalúa el comportamiento de las entidades en cada fotograma (separación de solapamientos, rebotes, límites de pantalla).
- **Módulo de Persistencia:** Maneja el flujo de I/O mediante System.IO para sincronizar la clase estática Config con el almacenamiento físico en el disco duro.

4.2 Metodología de Desarrollo

Para la realización de este proyecto se ha adoptado una metodología de desarrollo iterativa e incremental, fundamentada en los principios de las metodologías ágiles. Al tratarse de un proyecto individual, este enfoque ha permitido dividir la complejidad técnica del videojuego en ciclos de trabajo cortos y manejables, garantizando la obtención de un prototipo funcional y testeable al final de cada fase.

El ciclo de vida del desarrollo se ha estructurado en las siguientes iteraciones principales:

- **1. Fase de Análisis y Prototipado Visual:** Configuración inicial del entorno de trabajo en Visual Studio, definición de las dimensiones del lienzo (Config.cs), importación de *assets* gráficos (texturas de las setas) y diseño de la interfaz gráfica base (formularios de Windows Forms).
- **2. Fase del Motor Físico Básico:** Implementación del bucle de juego (Game Loop a 60 FPS) mediante temporizadores (Timer). Desarrollo de las físicas fundamentales: aplicación de vectores de gravedad, fricción horizontal y control de límites de la pantalla (rebotes en las paredes).
- **3. Fase de Lógica y Colisiones:** Diseño e implementación del algoritmo matemático central para la detección de colisiones circulares. Se programó la resolución de impactos elásticos (rebotes entre entidades de distinto nivel)

y el mecanismo de fusión (destrucción y creación de nuevas entidades de mayor nivel).

- **4. Fase de Integración de Reglas (Gameplay):** Incorporación del controlador del jugador (movimiento del ratón y *cooldown* de lanzamiento), sistema de acumulación de puntos y detección de la condición de derrota (Game Over) al superar la coordenada Y límite.
- **5. Fase de Persistencia y Pulido Audiovisual:** Integración de la librería System.IO para la escritura/lectura persistente del archivo local de récords. Inclusión de retroalimentación sonora mediante SoundPlayer para la música de fondo y los efectos especiales de fusión, además de la resolución de errores lógicos (control de clonación múltiple de entidades y solapamientos de audio).

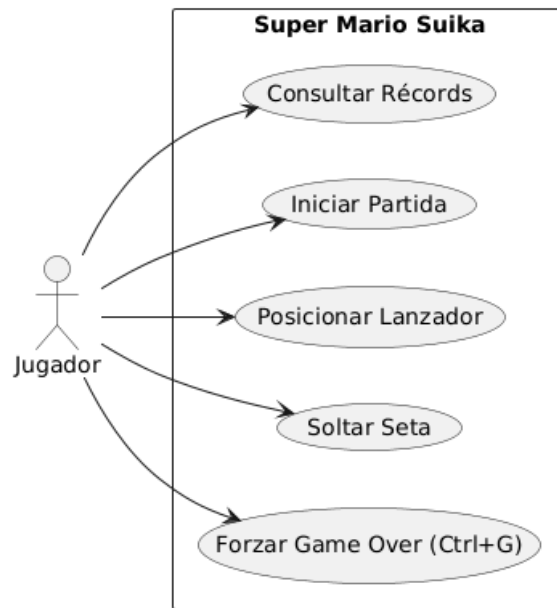
Durante todo el proceso se ha priorizado el uso de buenas prácticas de Programación Orientada a Objetos (POO), aplicando refactorizaciones continuas para asegurar que el código cumpliera con el principio de responsabilidad única, separando estrictamente la presentación visual de la lógica matemática del juego.

4.3. Arquitectura del sistema

Se ha empleado una arquitectura basada en el patrón de diseño Separación de Intereses. La capa visual (Formularios) no realiza cálculos físicos; delega estas tareas al GameService, inyectándole un modelo de datos (GameData) que actúa como única fuente de verdad. El lenguaje seleccionado es C# sobre el framework .NET (Windows Forms), utilizando su controlador SoundPlayer nativo para la capa de audio.

4.4. Diagramas de casos de uso

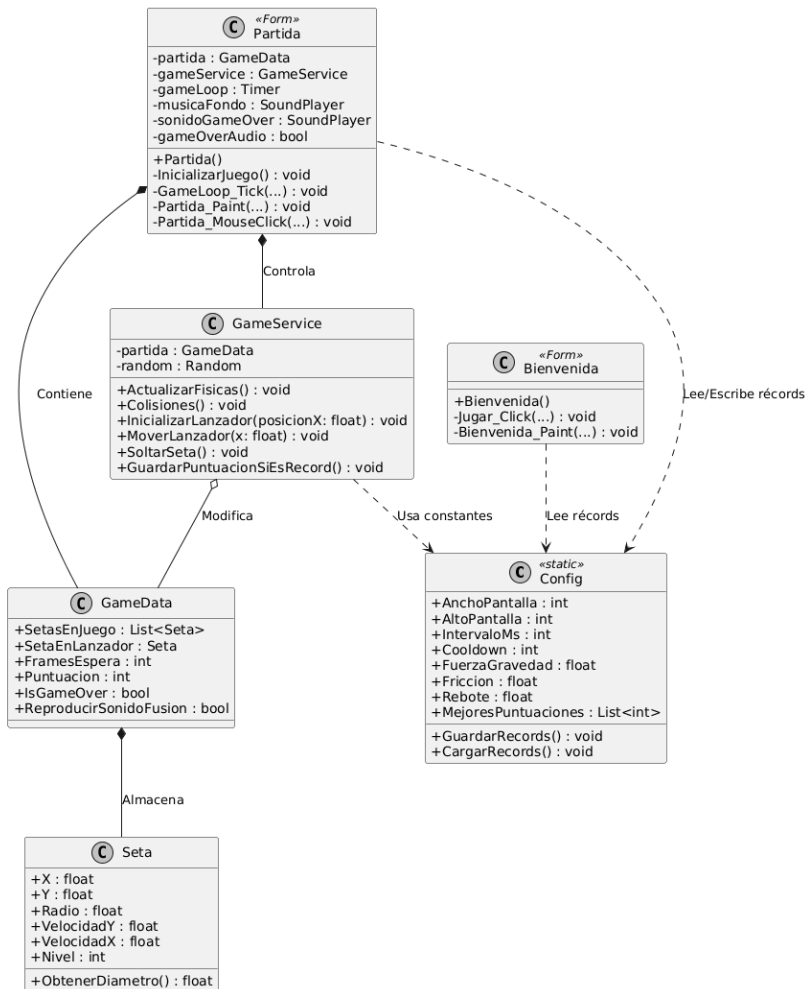
El sistema interactúa con un único actor principal (el Jugador), que desencadena los distintos eventos del juego.



- **Caso de Uso 1: Consultar Récords.** El jugador visualiza el Top 5 histórico al abrir la aplicación.
- **Caso de Uso 2: Iniciar Partida.** El jugador transita del menú principal al lienzo de juego.
- **Caso de Uso 3: Posicionar Lanzador.** El jugador mueve el ratón para desplazar horizontalmente la seta actual.
- **Caso de Uso 4: Soltar Entidad.** El jugador hace clic para que la física de gravedad afecte a la seta y caiga.
- **Caso de Uso 5: Forzar Game Over.** Mediante el atajo de desarrollador (Ctrl + G), el usuario finaliza la partida abruptamente para testear el guardado.

4.5. Diagramas de Clases

El diseño orientado a objetos del proyecto se estructura en clases con responsabilidades únicas y dependencias unidireccionales para evitar el acoplamiento.



- **Config (Clase Estática):** Contiene atributos públicos constantes (FuerzaGravedad, Friccion, AnchoPantalla) y la lista estática MejoresPuntuaciones. Métodos: GuardarRecords(), CargarRecords().
- **Seta:** Entidad básica. Atributos: X, Y, Radio, VelocidadX, VelocidadY, Nivel. Método: ObtenerDiametro().
- **GameData:** Contenedor de estado. Atributos: Puntuacion, IsGameOver, SetasEnJuego (Lista de objetos Seta), SetaEnLanzador (Objeto Seta).
- **GameService:** Lógica. Recibe un objeto GameData por inyección de dependencias en su constructor. Métodos: ActualizarFisicas(), Colisiones(), SoltarSeta().
- **Partida (Hereda de Form):** Interfaz gráfica. Contiene instancias de GameService, GameData y SoundPlayer.

- **Bienvenida (Hereda de Form):** Interfaz gráfica inicial. Depende de Config para mostrar los records.

4.6. Diseño de la Interfaz

Las interfaces gráficas (GUI) se han diseñado bajo el principio de usabilidad y *feedback* inmediato, utilizando GDI+ para el dibujado en tiempo real.

- **Pantalla de Bienvenida (Menú Principal):** Un lienzo estático que muestra el título del proyecto y centra la atención en el histórico de puntuaciones. Cuenta con un único botón de acción ("Jugar") claramente visible para iniciar el flujo principal.
- **Pantalla de Juego (HUD y Lienzo):**
 - **Área de interacción:** Lienzo con fondo negro de 500x800 píxeles.
 - **HUD (Heads-Up Display):** Etiqueta superior que actualiza la puntuación en tiempo real en la esquina superior izquierda.
 - **Línea de Peligro:** Trazo punteado blanco (Renderizado mediante la clase Pen) a 150 píxeles del margen superior que delimita visualmente la condición de derrota (Game Over).
 - **Feedback visual y sonoro:** Al activarse la condición de derrota, la interfaz bloquea el input del usuario, superpone un cartel de "GAME OVER" pre-renderizado y dispara una alerta sonora.

5. RESULTADOS E IMPLEMENTACIÓN

El desarrollo concluyó con un sistema robusto capaz de manejar de forma autónoma cálculos complejos. Como fragmento de código relevante para demostrar la lógica del sistema, destaca la implementación de colisiones para evitar el error lógico de "clonación múltiple":

```
C#
// Verificación de redundancia en colisiones (GameService.cs)
bool estanEliminadas = setasEliminar.Contains(setaA) ||
setasEliminar.Contains(setaB);

if (estanEliminadas == false)
{
    float distanciaAlCuadrado = (distanciaX * distanciaX) + (distanciaY *
distanciaY);
    // Lógica de colisión mediante Teorema de Pitágoras
}
```

Además, se resolvió con éxito el aislamiento del cierre de la ventana mediante el evento nativo `FormClosing`, garantizando que la grabación de datos en el disco duro ocurra de forma segura e invisible para el usuario sin importar el método de cierre.

6. CONCLUSIONES

6.1. Conclusiones del trabajo realizado

Se han cumplido íntegramente los objetivos planteados. El proyecto demuestra la viabilidad de crear experiencias interactivas complejas y fluidas prescindiendo de motores de terceros. El sistema de físicas maneja con soltura la transferencia de energía y la colisión de decenas de elementos simultáneos a 60 FPS, y la persistencia de datos añade un alto valor de rejugabilidad al sistema.

6.2. Trabajos futuros

El diseño modular del código permite anticipar mejoras futuras que se han descartado para esta primera fase, tales como:

- **Sistema de Previsualización:** Implementar un indicador visual (Next) que anticipe al jugador el tipo de seta que recibirá en el siguiente lanzamiento.
- **Efectos de Partículas:** Añadir una pequeña animación o explosión gráfica en el instante exacto en el que dos entidades se fusionan.

7. BIBLIOGRAFÍA / WEBGRAFÍA

- Microsoft. (2024). System.Drawing Namespace. Documentación oficial de .NET.
<https://learn.microsoft.com/es-es/dotnet/api/system.drawing>
- Microsoft. (2024). SoundPlayer Class. Documentación oficial de .NET.
<https://learn.microsoft.com/es-es/dotnet/api/system.media.soundplayer>